

Exemple, LeSas (v2)

Fabrice.Kordon@lip6.fr



Variante par rapport à la version 1

2

Objectif, une seule primitive...

-  entrerDansSas() + sortirDuSas()
-  passerLeSas()
-  Mais sinon, le problème reste le même...

Pourquoi?

-  Discussion intéressante sur le fonctionnement d'un moniteur de Hoare

La classe Passagers (v2)

3

```
class Passenger2 implements Runnable {
    private Sas2 s;
    private int monId;
    private static final Object mutex = new Object();
    private static int cpt = 1;

    Passenger2 (Sas2 s) {
        synchronized (mutex) {
            monId = cpt++;
        }
        this.s = s;
    }

    private void trace (String m) {
        System.out.println ("Passager " + monId + " : " + m);
        Thread.yield(); // rendre la commutation possible
    }

    public void run() {
        try {
            this.trace ("prêt à entrer");
            s.passerLeSas(monId);
            this.trace ("sorti");
        }
        catch (InterruptedException e) {
            this.trace ("problème dans le sas " + e);
        }
    }
}
```

La classe Sas (méthode de synchronisation, v2)

4

```
public void passerLeSas (int qui) throws InterruptedException{
    l.lock();
    try {
        while (!remplir) { // gestion de l'entrée
            entrer.await();
        }
        contenu--;
        if (contenu == 0) {
            System.out.println (" " + qui + " dedans + ferme l'entrée");
            remplir = false;
            sortir.signalAll(); // sas plein, réveil coté sortie
        } else {
            System.out.println (" " + qui + " dedans");
        }
        while (remplir) { // gestion de la sortie
            System.out.println (" " + qui + " devant la sortie");
            sortir.await();
        }
        contenu++;
        if (contenu == capacite) {
            System.out.println (" " + qui + " ferme la sortie");
            remplir = true;
            entrer.signalAll(); // sas vide, réveil coté entrée
        } else {
            System.out.println (" " + qui + " sort");
        }
    }
    finally {
        l.unlock();
    }
}
```

La classe Sas (méthode de synchronisation, v2)

4

```
public void passerLeSas (int qui) throws InterruptedException{
    l.lock();
    try {
        while (!remplir) { // gestion de l'entrée
            entrer.await();
        }
        contenu--;
        if (contenu == 0) {
            System.out.println (" " + qui + " ferme la sortie");
            remplir = false;
            sortir.signalAll(); // sas vide, réveil coté entrée
        } else {
            System.out.println (" " + qui + " sort");
        }
        while (remplir) { // gestion de la sortie
            System.out.println (" " + qui + " devant la sortie");
            sortir.await();
        }
        contenu++;
        if (contenu == capacite) {
            System.out.println (" " + qui + " ferme la sortie");
            remplir = true;
            entrer.signalAll(); // sas vide, réveil coté entrée
        } else {
            System.out.println (" " + qui + " sort");
        }
    }
    finally {
        l.unlock();
    }
}
```



À la manière Java
signalAll()

Quelques exécutions

5

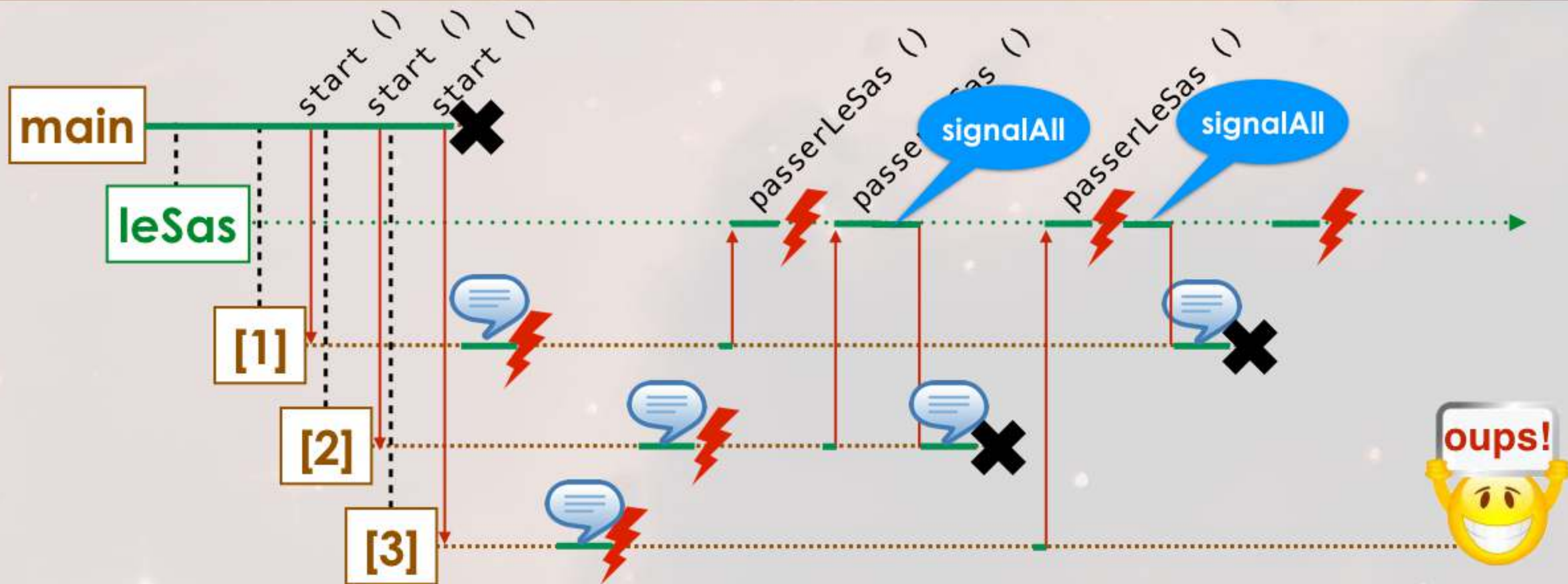
```
$ java LeSas2 2 3
Passager 1 : prêt à entrer
Passager 2 : prêt à entrer
Passager 3 : prêt à entrer
  1 dedans
  1 devant la sortie
  2 dedans + ferme l'entrée
  2 sort
Passager 2 : sorti
  1 ferme la sortie
Passager 1 : sorti
  3 dedans
  3 devant la sortie
```

^C

```
$ java LeSas2 2 4
Passager 1 : prêt à entrer
Passager 3 : prêt à entrer
Passager 4 : prêt à entrer
  1 dedans
Passager 2 : prêt à entrer
  1 devant la sortie
  3 dedans + ferme l'entrée
  3 sort
Passager 3 : sorti
  1 ferme la sortie
Passager 1 : sorti
  4 dedans
  4 devant la sortie
  2 dedans + ferme l'entrée
  2 sort
Passager 2 : sorti
  4 ferme la sortie
Passager 4 : sorti
```

Chronogramme d'une exécution

6

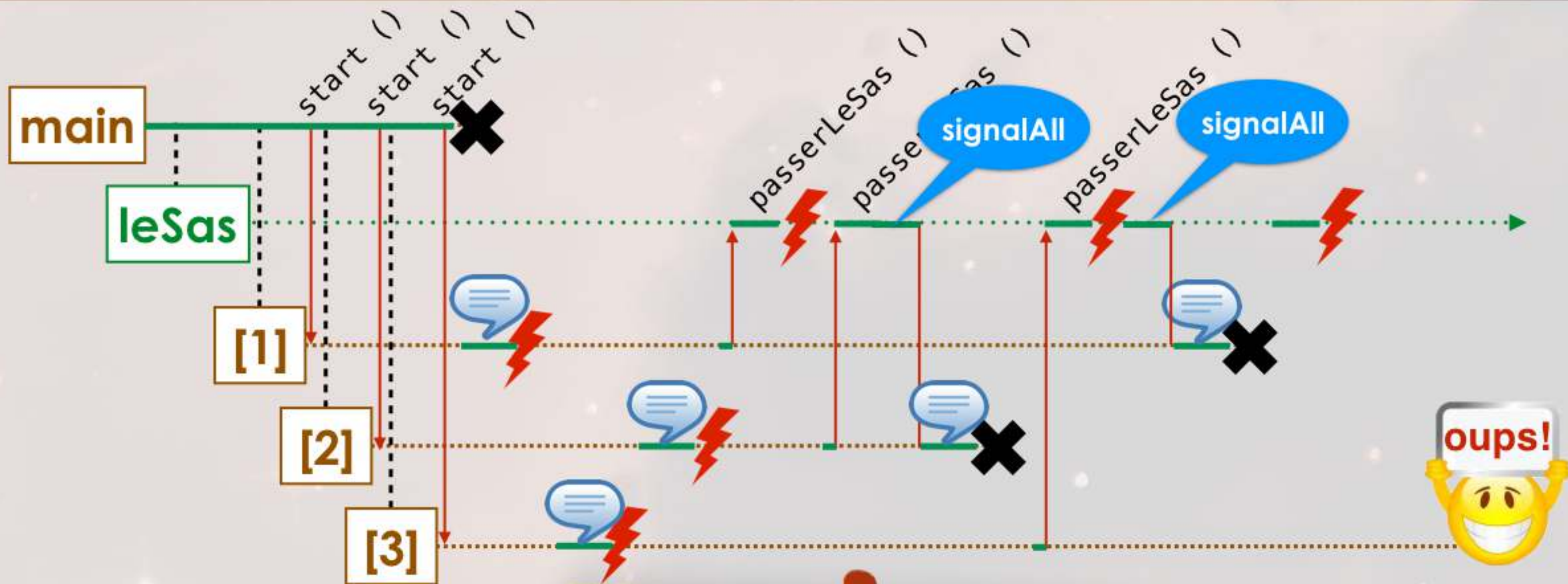


```
$ java LeSas2 2 3
Passager 1 : prêt à entrer
Passager 3 : prêt à entrer
Passager 2 : prêt à entrer
1 dedans
1 devant la sortie
2 dedans + ferme l'entrée
2 sort
Passager 2 : sorti
1 ferme la sortie
Passager 1 : sorti
3 dedans
3 devant la sortie
```

^C

Chronogramme d'une exécution

6



On a toujours ce blocage

Belle illustration...

Un seul fil actif dans le moniteur

```
$ java LeSas2 2 3
Passager 1 : prêt à entrer
Passager 3 : prêt à entrer
Passager 2 : prêt à entrer
1 dedans
1 devant la sortie
2 dedans + ferme l'entrée
2 sort
Passager 2 : sorti
1 ferme la sortie
Passager 1 : sorti
3 dedans
3 devant la sortie
```

^C